

SURGEON PREFERENCE CARD PLATFORM

A modular, cloud-ready system for managing surgeon preference cards with and without EPR integration.

1. Overview

Most UK theatres still rely on paper preference books or rigid EPR modules that don't scale across:

multiple specialities

multiple surgeons

multiple specialities

And does not cater for each surgeon having many procedures complex instrument/consumable requirements.

This platform provides a modern, structured, version-controlled alternative with two parallel pipelines:

a. A Standalone Pipeline with no EPR integration

Streamlit UI



Validation



Database



Kardex PDF

b. An EPR-Integrated Pipeline:

Simulated Cerner/EPIC feed



Adapter



Database



Kardex PDF

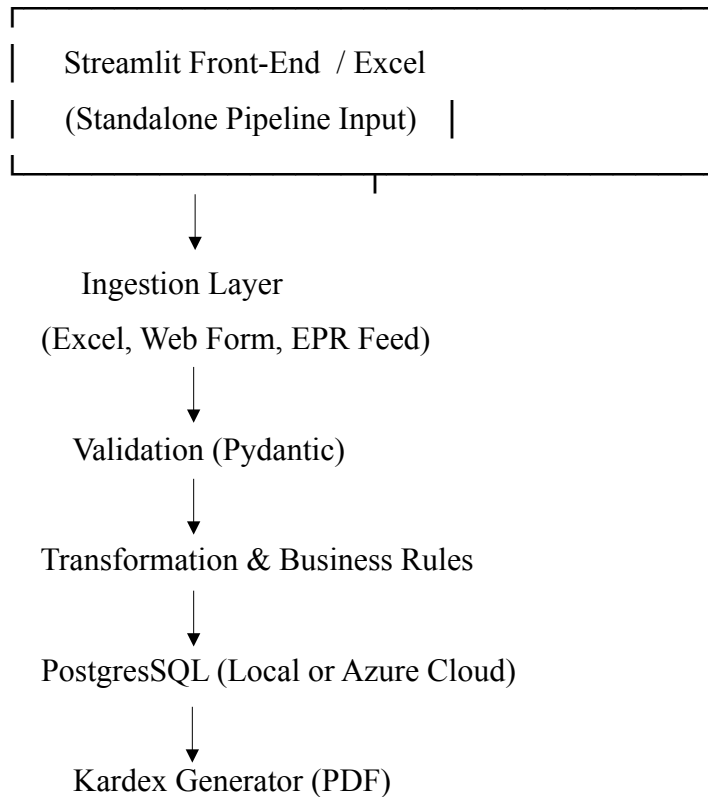
Both pipelines share the same domain models, validation, transformation logic, and output generator.

2. Goals:

- Provide a single source of truth for surgeon preference cards
- Allow staff to update preferences using a simple Streamlit interface
- Validate and normalise data using Pydantic models
- Store structured data in PostgreSQL (local or Azure)
- Generate Kardex PDFs in a consistent format
- Support future EPR integration via an adapter layer
- Run locally for development and in Azure for production

3. High-Level Architecture:

Code



4. Pipeline A — Standalone (Streamlit Front-End)

4.1 Workflow

Code

Streamlit Web Form



Ingestion Layer (JSON payloads)



Pydantic Validation



Transformation Layer



PostgreSQL



Kardex PDF Generator

4.2 Streamlit UI Features

- Select surgeon
- Select procedure
- Add instruments
- Add consumables
- Add positioning, anaesthetic notes, special instructions
- Save/update preference card
- Generate Kardex PDF

Streamlit sends structured JSON to the ingestion layer.

4.3 Local vs Azure Behaviour

Component	Local Mode	Azure Mode
Streamlit	Local Host: 8501	Azure App Service
Database	Docker Postgres	Azure PostgreSQL Flexible Server
Storage	Local File System	Azure Blob Storage
Secrets	.env	Azure Key Vault
Logging	Local JSON Logs	Azure Application Insights

Switch modes via:

- Yaml
- Environment: local # or "azure"

5. Pipeline B — EPR-Integrated (Simulated Cerner/EPIC Feed)

5.1 Workflow

Code

Simulated EPR Feed (JSON)



EPR Adapter (mapping + validation)



Transformation Layer



PostgreSQL



Kardex Generator

5.2 Why an EPR Adapter?

Every EPR system structures preference data differently:

Cerner → flat lists

EPIC → nested objects

CareFlow → CSV-style extracts

The adapter converts any EPR format into the internal domain model, ensuring consistency.

6. Detailed System Components

6.1 Domain Layer

Defines the canonical data model:

- Surgeon
- Procedure
- Instrument Set
- Consumables
- Positioning
- Anaesthetic Notes
- Special Instructions
- Version Metadata

6.2 Ingestion Layer

Handles input from:

- Streamlit (JSON)
- Excel uploads
- EPR feed (simulated)
- 6.3 Validation Layer
- Uses Pydantic to enforce:
 - required fields
 - correct data types
 - valid enums
 - business rules

6.4 Transformation Layer

- Normalises and enriches data:
- standardises instrument names
- applies speciality-specific rules
- ensures consistent structure

6.5 Load Layer

- Stores validated data in:
 - Local Postgres (Docker)
 - Azure PostgreSQL Flexible Server

6.6 Output Layer

Generates:

- Kardex PDFs using ReportLab
- JSON exports (optional)

7. Folder Structure

Code

surgeon_preference_platform/

```
|
|— domain/
|   |— models.py
|   |— enums.py
|   └— validators.py
|
|— ingestion/
|   |— streamlit_ingestor.py
|   |— excel_ingestor.py
|   └— epr_ingestor.py
|
|— transform/
|   |— normalise.py
|   └— business_rules.py
|
|— load/
|   |— postgres_loader.py
|   └— kardex_generator.py
|
|— ui/
|   └— streamlit_app.py
|
|— config/
|   └— settings.yaml
```

```
|   └─ logging.yaml
|
|   └─ tests/
|
|   └─ README.md
```

8. Tools & Technologies

8.1 Core Python Stack

- Pydantic Models
- SQLAlchemy Core
- psycopg2
- ReportLab
- PyYAML

Loguru

8.2 Front-End

- Streamlit

8.3 Local Development

- Docker Desktop
- Makefile
- .env secrets
- pytest

8.4 Azure Cloud

- Azure App Service
- Azure PostgreSQL Flexible Server
- Azure Blob Storage
- Azure Key Vault
- Azure Monitor / App Insights
- GitHub Actions or Azure DevOps

9. Roadmap

- ☐ Implement domain models
- ☐ Build Streamlit UI
- ☐ Create Excel + JSON ingestion
- ☐ Build EPR adapter
- ☐ Implement transformation logic
- ☐ Build Postgres loader
- ☐ Generate Kardex PDFs
- ☐ Add versioning
- ☐ Add tests
- ☐ Add CI/CD
- ☐ Deploy to Azure
- 10. Status
- Active development.

This README defines the architecture and direction for the full implementation.